

MLM2Vec-Driven Multimodal Toxicity Detection

Sameer Navuduri

*Amrita School of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India
sam.navuduri@gmail.com*

Lakshmi Venkat Ramisetty

*Amrita School of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India
ramisettylakshmivenkat@gmail.com*

Sai Raghavendra Rangisetty

*Amrita School of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India
sairaghavendra179@gmail.com*

Hima Yalavarthi

*Amrita School of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India
yalavarthihima@gmail.com*

Dr. Premjith B

*Amrita School of Artificial Intelligence
Amrita Vishwa Vidyapeetham
Coimbatore, India
b_premjith@cb.amrita.edu*

Abstract—Toxicity detection is the identification of toxic, abusive, or offensive content in digital communication. Multimodal toxicity detection is the identification of multiple data modalities such as text, audio, and video to improve the accuracy and contextual understanding of toxic behavior. In video data, it is identifying what is being spoken, facial expressions, and visual content in addition to text transcripts. Code-mixed Hindi-English content is even more difficult with non-standard grammar and spelling, and code-switching. Toxicity detection in such multimodal, code-mixed content requires models with the capability to process and comprehend multiple signals simultaneously while dealing with linguistic complexity, and therefore it is a core and less-explored research task. The proposed method gives accuracies of 72% in the task of toxicity detection, 79% in severity detection and 74% in the task of sentiment classification. It is able to demonstrate its use for nuanced context capture.

Index Terms—multimodal, toxicity detection, code-mixed, decoder.

I. INTRODUCTION

Toxicity detection refers to the identification of anti-social behaviours [22] on online platforms such as cyberbullying, threats and hate speech. With the rise in the numbers of users, the amount of data increased drastically. With the kinds of features (audio, text and video) available, it is essential to use every piece of information we have, to discern the intentions and potential harm behind the online activity. The inclusion of video data brings the task of training the model on the facial expression, spoken language and the tone. Code-mixed Hindi-English data refers to the combination of Hindi and English words within a single sentence or statement, common in Indian online language [25]. Detoxification of code-mixed Hindi-English data is becoming increasingly important with greater use on video and social media platforms [26]. Multimodal analysis of code-mixed Hindi-English data yields a richer user intent representation, improving implicit or culturally bound toxicity understanding.

Multimodal code-switching Hindi-English datasets are especially difficult with grammar inconsistency, spell variation,

and transliteration. There are difficulties in feature alignment between modalities and temporal dependencies within the video. Recent state-of-the-art multimodal toxicity detection pipeline extracts features for every modality through mechanisms such as wav2vec2.0 for audio, Vision Transformers [10] for video, and transformers such as BERT for text [5] [8]. These characteristics are aggregated and handed over to predictors such as Random Forest or neural networks for making predictions. Textual characteristics for code-mixed Hindi-English data are normally obtained by means of multilingual transformers or LLM embeddings [7] [25]. Aggregate feature representations are subsequently trained over by classifiers to predict toxicity. Most of them are, however, highly dependent upon encoder-based models and do not utilize the full potential of decoder-only language models. They also do not clearly examine the impact of causal masking or layer-wise attention on performance.

The objective of our research was to investigate the use of decoder-only models for the task of multimodal toxicity detection. Considering the objective, two frameworks were suggested. One framework uses the novel LLM2Vec methodology. This method involves the modification of the decoder model to enable bidirectional attention. The second framework involves the use of the original decoder model to generate the textual embedding.

While previous works mostly focused on using encoder models and machine learning models [6] [14], this work uses a decoder model for the task. Additionally, it provides a comparison via scores to learn if using bidirectional attention is helpful or not. It also provides us with a comparison among the machine learning models for the classification of the multimodal embedding. This work demonstrates an efficient framework to detect toxicity using all three modes and the usefulness of the generated multimodal embedding with the help of t-SNE plots.

The first observation derived from the obtained results is that the LLM2Vec framework has achieved more accuracy in classifying it as offensive and non-offensive. However, for

the task of fine-grained classification (offensiveness level), the MLM2Vec framework has performed much better than the LLM2Vec framework. This shows us that while enabling bidirectional attention helps achieve better results in higher-level classification, the causal attention mechanism is more helpful in understanding the nuances. This framework is well supported by the Random Forest classifier, which handles the non-linearity of the generated embeddings, which can be seen in the t-SNE plots.

The chapter follows the given structure: Section II focuses on the literature that helped define our objectives and aided our work as reference materials. The pros and cons of every cited research are covered, and how they are relevant to this work are explained. Section III explores the algorithms that were utilized to fuel our work and attain the current results. It dives into the details and explains how the various parameters are relevant to our research. Section IV lists out the methodology and its nuances. The feature generation, embeddings fusion, and the testing performed with the Machine Learning classifier is explained in great detail with the help of flowcharts so that the reader can visualize the process and understand how it would be helpful to them. Section V is about the results generated from our crafted framework and its interpretation. It will be used to explain how the model works in this scenario. Section VI will conclude our work by summarizing our overall work and mapping out the scope for future research based on our work.

II. LITERATURE SURVEY

Initial work in toxicity detection was based on text content analysis with classic machine learning models such as support vector machines (SVMs) and random forests. Schmidt and Wiegand (2017) presented an extensive overview of hate speech detection by natural language processing, emphasizing the power of word embeddings and deep learning-based models [1]. Subsequently, Fortuna and Nunes (2018) presented different supervised and unsupervised techniques for hate speech, showing that deep neural networks remarkably outperform standard methods [2]. The release of transformer models as the BERT [4], spearheaded text-based toxicity identification by allowing contextual comprehension and transfer learning opportunities [3]. The models had better classification results in numerous languages but were mostly trained with monolingual data, resulting in the models being somewhat weaker for code-mixed expressions [23].

With an increasing amount of audio and visual content being consumed through online channels, researchers have investigated multimodal toxicity detection for improving classification. The ToxicVidLM dataset (Maity et al., 2024) was proposed to enable toxicity detection in multimodal environments, specifically for code-mixed Hindi-English utterances. It contains speech, visual, and textual features, and is thus a valuable resource for improving research in this area [5]. Feature extraction and fusion methods for multimodal learning have been studied in some works. In a work on multimodal transformers, it was shown that the fusion of text, speech,

as well as visual embeddings, can enhance performance in toxicity detection compared to uni-modal methods (Kiela et al., 2020) [6]. Multimodal fusion, however, is still a challenging problem owing to discrepancies in data alignment and representation.

In 2024, researchers from the University of Zagreb [13] presented a new method for attention modulation in transformer models. Through the removal of layer-wise causal masks, the approach enables models to process longer-range dependencies and contextual relationships more flexibly. This bears relevance for toxicity detection, especially where context has to be analyzed beyond a fixed left-to-right dependency order.

The works on Laughing Heads [27] investigated multimodal sentiment and toxicity classification through the examination of facial expression and patterns of laughter in combination with speech and text content. The research emphasized the use of humor and sarcasm in toxic communication, proving that the addition of facial expressions improves the accuracy of toxicity classification models.

One of the main contributions of our work is using LLM2Vec, a method for domain adaptation of large language models (LLMs) to produce embeddings for various modalities. LLM2Vec was initially proposed for textual embeddings but has since found extension to multimodal applications, proving to be effective in numerous tasks [6]. Recent breakthroughs in self-supervised learning have greatly enhanced speech representation learning. Models such as the Wav2Vec 2.0 [8] and HuBERT [9] enable efficient audio feature extraction, improving speech-based toxicity detection [7]. These methods enable improved generalization for low-resource languages, hence making them very useful for Hindi-English code-mixed speech [24] [23]. For video-based toxicity detection, transformer-based models like Vision Transformers (ViT) [10] and CLIP [11] have been employed to produce visual embeddings that match textual ones. These models have shown strong performance for tasks involving multimodal reasoning and thus are pertinent to toxicity detection. A primary challenge of multimodal learning is incorporating multiple streams of data in an effective manner. Current work on feature fusion can be generally classified into early fusion, late fusion, and hybrid fusion methods. Baltrušaitis et al. (2018) presented an extensive overview of multimodal fusion methodologies, highlighting their use in deep learning [11]. The present work investigates various fusion mechanisms for improving multimodal toxicity classification. The body of work on toxicity identification has grown exponentially, from text-only methods to sophisticated multimodal methods that use speech and video inputs. Although earlier studies offer valuable insights, the absence of strong solutions for code-mixed Hindi-English toxicity detection is an area of lassitude to which our task is to contribute. By taking advantage of transformer-based embeddings, self-supervised learning of speech, and vision transformers, we introduce an efficient multimodal toxicity detection framework.

III. BACKGROUND

In this section, we will go over the preview of the algorithms and models underlying the methodology that are being used. They establish an important foundation for the methods that are being used.

1) *Transformers*: Transformers [1] are created to process sequences as inputs, so they can generate the corresponding sequence as output. They are highly versatile, allowing them to be used in multiple fields. The BERT model released by Google proved to be a major milestone in the world of transformers, evident from how it is prevalently used for embeddings generation upto this day [4]. The transformer architecture commonly consists of the encoder-decoder modules that are used to learn the input representation and later generate the answers, analyze the sentiment, or summarize the given input. All of this stems down to the central idea of **attention**.

Attention is used to understand the relevance of every other token in the model's vocabulary (essentially, all the possible tokens it can know) with respect to the current token that is being examined. Maps a query and the key-value pairs to the corresponding output. Although various updates to this mechanism have been introduced, the most basic and common mechanism is the **scaled dot product attention**. They are computed using:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right)V$$

2) *LLM2VEC*: LLM2Vec is an approach that converts decoder-only LLMs to text encoders by enabling bidirectional attention, and then performing masked next-token prediction and contrastive learning. It can support several models such as Llama-3 and Qwen-2 with both supervised and unsupervised training modes. The framework enables flexible sequence encoding and pooling operations. One can load pretrained models with LoRA weights tuned for various tasks. It produces effective embeddings with mean pooling as a default but has support for other pooling approaches. LLM2Vec is open-source and comes with PyPi and Hugging Face support.

3) *Autoregressive Models*: Decoder-only transformer models, such as the GPT [18], [19], are used to generate text based on the given context vector derived from the encoder output. LLaMA (Large Language Model Meta AI) is an autoregressive model based on the *GPT-style architecture*. It follows the standard transformer design but with optimizations such as *rotary positional embeddings* [20] and *grouped-query attention* [28] for efficiency. Below are key equations explaining its working:

- 1) **Token Embedding & Input Representation** Each input token x_i is first mapped to a high-dimensional vector through an embedding layer:

$$h_0 = Ex$$

where E is the learned embedding matrix.

- 2) The self-attention mechanism LLaMA uses **multi-head self-attention (MHSA)** to capture dependencies

between tokens. For each head i , the attention scores are calculated as:

$$Q = XW_Q, \quad K = XW_K, \quad V = XW_V$$

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

where Q, K, V are the **query, key and value matrices**, and d_k is the dimensionality of each head.

- 3) **Grouped-Query Attention (GQA) Optimization**: To improve efficiency in multi-head attention, **GQA** reduces the number of keys and values per group, allowing faster inference:

$$\tilde{K} = \text{Concat}(K_1, K_2, \dots, K_g), \quad \tilde{V} = \text{Concat}(V_1, V_2, \dots, V_g)$$

where g is the number of key-value groups, reducing memory access overhead.

- 4) **Feedforward Network (FFN) Transformation** Each transformer block also includes a **position-wise feedforward network (FFN)**:

$$\text{FFN}(h) = \sigma(hW_1 + b_1)W_2 + b_2$$

where W_1, W_2, b_1, b_2 denote the learned parameters, and σ is a non-linearity (typically **SiLU**).

- 5) **Rotary Positional Embeddings** [20]: Instead of absolute positional encodings, LLaMA 2 uses *RoPE*, which encodes position into attention scores using a rotation matrix:

$$\text{RoPE}(q, k) = \cos(\theta_p)q + \sin(\theta_p)k$$

where θ_p is a position-dependent phase shift, enabling better long-range interactions.

- 6) **Autoregressive Language Modeling Objective** LLaMA is trained using *causal masked self-attention*, where each token attends only to past tokens. This means that they can only look in one direction.

$$p(x_t|x_1, \dots, x_{t-1}) = \text{softmax}(W_o h_t)$$

where W_o maps hidden states to vocabulary logits. The loss function is the **negative logarithmic likelihood (NLL)** over the data set:

$$\mathcal{L} = -\sum_{t=1}^T \log p(x_t|x_{<t})$$

This ensures that the model learns to predict the next token in a sequence.

These optimizations make LLaMA *faster and more memory efficient* while preserving the state-of-the-art performance in language modeling tasks.

4) *Video Transformers*: CNNs and Vision Transformers (ViTs) are state-of-the-art performers in the field of image processing. While CNNs effectively capture patterns, ViTs [10] process the whole image. However, given the computational cost, they need improvement in terms of their efficiency. The MobileViT [15] is a combination of CNNs and ViTs, detecting patterns while capturing the global context of the image. They require lesser number of parameters, as they are especially optimized for mobile devices. To bring the best of both the worlds, the MobileViT model replaces some of the CNN layers with small transformer blocks. The process starts with regular convolutions to obtain low-level features, followed by the feature map being split into small patches and flattened. The patches are then fed through a transformer encoder, where self-attention is used to capture global dependencies within the image. The processed patches are then reshaped back into a structured feature map and fed through further convolutions for final classification. This makes these models well suited for real-world applications and helps researchers overcome the real world constraint of training on extensive datasets.

5) *Self-Supervised Learning for Speech Representations*: The Wav2Vec 2.0 model [8], introduced by Facebook, significantly improves automatic speech recognition by developing feature-rich audio representations from raw audios. It begins by creating a temporary representation, z , with the help of a CNN-based feature extractor. This representation is fed into an encoder model that captures the relation across different parts of the entire audio signal. Following this, the model uses self-supervised learning [16], by randomly masking parts of the representation z and enabling the model to predict the missing features. The loss function used to separate the ground truth masked embedding q_t from the distractors Q_t :

$$L = - \sum_t \log \frac{e^{sim(q_t, k_t)/\tau}}{\sum_{k \in Q_t} e^{sim(q_t, k)/\tau}}$$

where $sim()$ indicates any similarity function like the cosine similarity, k_t denotes true latent representation and τ represents the temperature. All this pre-training is done with the help of unlabeled data. Once the pre-training is over, the Wav2Vec2 model is fine-tuned using smaller labeled datasets.

6) *Synthetic Minority Over-sampling Technique (SMOTE)*: This method is widely used to deal with class imbalance in the given data. The lack of data is balanced by populating the data with artificially generated data points representing the minority class. To do so, we iterate every existing data point x , their nearest neighbor x_m , and take a point that lies between them using the equation:

$$x_{new} = x + \lambda(x_m - x)$$

This method not only serves its purpose to help handle the class imbalance, but also generate perturbed data that can help the model learn without being too biased. However, this method cannot be used for imbalanced data and complex distributions.

7) *Principal Component Analysis (PCA)*: It is a technique used to reduce the dimensions, while preserving the essential features of the given data using its eigenvalues and eigenvectors. It is started by calculating the covariance matrix of the input, and deriving its eigenvalues and eigenvectors. Given the data matrix X , we normalize it. Then we take matrix W , which consists of the top k eigenvectors. We then derive the projected from the eigenvectors corresponding the top k eigenvalues.

$$Z = XW$$

PCA reduces the dimensions of the embeddings to make it faster for processing. It has been used to reduce overfitting in machine learning models.

8) *Gated Fusion*: To identify toxic content, our work adopted a hybrid approach that reconciled information from text, audio, and video modalities. This approach combined feature-level and decision-level fusion. For feature-level fusion, we formed a single, integrated representation by concatenating the embedded representations. These representations were produced by LLM2Vec for the text modality, Wav2Vec2.0 for the audio modality, and Vision Transformers (ViT) for the video modality. The mathematical representation of this fusion can be stated as follows:

$$F = f(T, A, V) = W_t T + W_a A + W_v V$$

Here, T , A , and V refer to the feature vectors obtained from the text, audio, and video modalities, respectively, and W_t , W_a , and W_v are the weight matrices learned. For decision-level fusion, we used a weighted ensemble approach by fusing the predictions of single modality-specific classifiers. The final classification decision was subsequently made through a weighted summation of these single outputs. The proposed hybrid fusion methodology efficiently combined the strengths of both integration methods to enable a strong multimodal classification of toxicity while retaining each modality's unique characteristics.

9) *Random Forest Classifier*: Random Forest is an ensemble learning algorithm that utilizes multiple decision trees and combines their output to obtain higher accuracy and minimize overfitting. It works by training various decision trees on different subsets of the data using bootstrapping aggregation (bagging) with each tree trained on a unique sample of data with replacement. Also, for every node split, a random subset of the features is taken instead of employing all the features available, thus increasing model diversity. The last prediction is arrived at by majority voting for classification problems or prediction averaging for regression problems. Algebraically, from a collection of decision trees $h_1(x), h_2(x), \dots, h_n(x)$, the last output $H(x)$ is calculated as:

$$H(x) = \frac{1}{n} \sum_{i=1}^n h_i(x) \text{ for regression}$$

or

$$H(x) = \text{mode}(h_1(x), h_2(x), \dots, h_n(x)) \text{ for classification}$$

where $h_i(x)$ denotes the prediction of the i -th tree. The ensemble of many trees greatly enhances generalization and noise robustness, making Random Forest a very strong and popular algorithm for structured data classification and regression problems.

IV. METHODOLOGY

A. Overview

Here, we explain the techniques utilized for multimodal toxicity classification with code-mixed Hindi-English sentences. Our system combines text, audio, and video modalities by extracting strong feature representations from each of them and then combining them into a single embedding to classify. The overall pipeline (depicted in Figure 4.1) consists of data preprocessing, feature extraction for individual modalities, multimodal fusion through a gated mechanism, and supervised classification. The following sections examine each of these components in detail, working as explained in the previous chapter.

B. Data Preprocessing

C. Dataset Description

We use the ToxicVidLM dataset, which comprises 4021 utterances of code-mixed Hindi-English utterances [5]. Each sample includes three modalities:

- Text: Representing the spoken content.
- Audio: Captured from the original speech.
- Video: Recorded frames containing visual cues (e.g., facial expressions).

The target columns are: "**offensive**" (yes/no), "**offensiveness level**" (high, medium, low), and "**sentiment**" (positive, neutral, negative).

D. Data Validation

Random sampling is used to create about 200 samples in each category, to avoid bias in the machine learning classifier. This balanced dataset is used for training and evaluation.

E. Modality-Specific Preprocessing

F. Text

- Text data is cleaned by converting to lowercase and removing non-alphabetical characters.
- Tokenization is performed using a modified tokenizer for our LLaMA 2 model. We resolve padding issues by setting the pad token to the EOS token

G. Audio

- Audio files are resampled to 16 kHz using librosa, which is required for processing by Wav2Vec 2.0.
- The raw waveform is processed by an audio processor to create inputs for Wav2Vec 2.0, which outputs hidden state representations.

H. Video

- Videos are processed by extracting frames using a frame-capturing utility.
- Each frame is passed through a lightweight MobileViT model to extract visual embeddings.
- The frame-level embeddings are aggregated (via sum-pooling) into a single video representation.

I. Feature Extraction with MLM2Vec

J. Text Embeddings with Llama 2

A 4-bit precision-reduced quantized version of the LLaMA 2 7B model is used. The model, as a decoder-only transformer, is modified to generate sentence embeddings based on the representation from the final token. In particular, for every text sample, we carry out the following:

- 1) Tokenize the input text using proper padding.
- 2) Perform a forward pass using the model set to output hidden states.
- 3) Take the last hidden layer's output and utilize the last token's embedding as the sentence representation:

$$h_{\text{sentence}} = h_T^{(L)}$$

where $h_T^{(L)}$ represents the last token's hidden state at the end layer L .

K. Audio Embedding with Wav2Vec 2.0

For the audio, we employ the pre-trained Wav2Vec 2.0 model. Here is how it works:

- 1) Load and resample the audio signal to 16 kHz.
- 2) Process the waveform with the audio processor to generate model inputs.
- 3) Extract the hidden states of the last layer.
- 4) Compute mean pooling over the time dimension in order to get a fixed-size audio embedding:

$$a = \frac{1}{T} \sum_{t=1}^T h_t^{\text{audio}}$$

where h_t^{audio} is the hidden state at time t .

L. Video Embedding with MobileViT

Video processing includes transforming videos into a series of frames and deriving features from every frame using MobileViT:

- 1) Pull frames from the video with a frame capture tool.
- 2) For every frame, feed it into the MobileViT model to get a frame-level embedding.
- 3) Average the frame embeddings by summing or averaging them to get a single video embedding:

$$v = \frac{1}{N} \sum_{i=1}^N f_i$$

where f_i is the i th frame embedding and N is the number of frames.

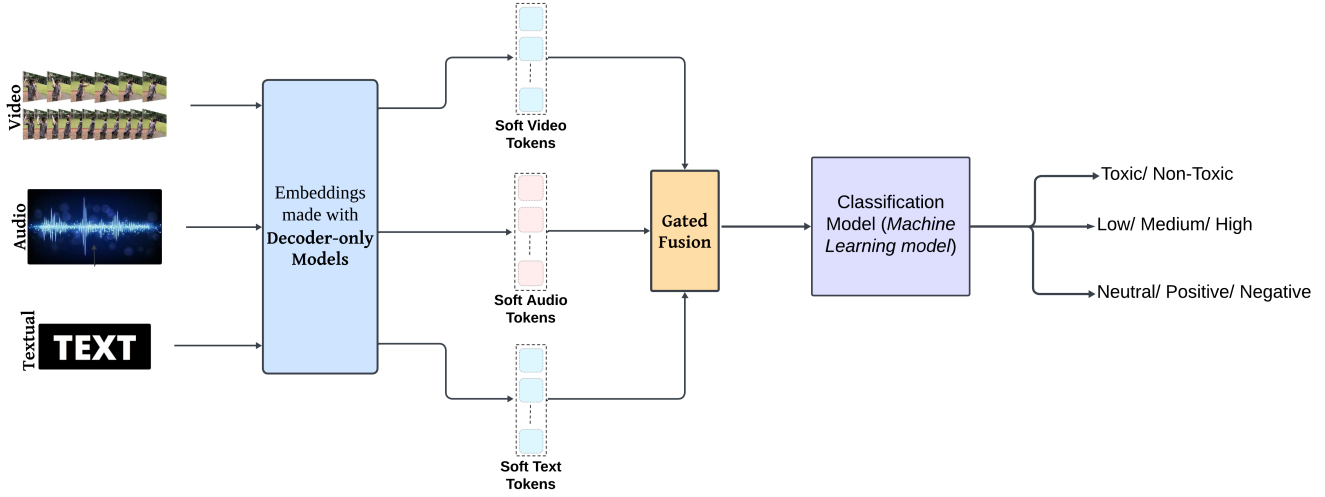


Fig. 1. Methodology to evaluate the MLM2Vec framework

M. Feature Extraction with LLM2Vec

The block diagram charted in Figure 4.2 summarizes our process followed to extract and evaluate the embeddings obtained with the LLM2Vec framework.

N. Text Embeddings using LLM2Vec

Hugging Face imports the LLM2Vec model (McGill trained Mistral-7B-Instruct-v2 model), if GPU support exists. The model's **encode()** function is utilized to convert the tokenized preprocessed input text data into embeddings in vector format. The embeddings are stored as a NumPy array in a *.npy* file for further project processing. If feasible, PEFT-tuned is used to create embeddings quickly. This method facilitates the processing of text data in an efficient manner, aids downstream tasks with best performance, and employs deep learning methods for representation learning.

O. Audio Embedding with LLM2Vec

The audio waveforms are processed using the Whisper model [29] to generate numerical vectors. After converting these vectors to string, they are fed to the LLM2Vec model. Using its **encode()** function, the LLM2Vec model generates embeddings for the corresponding audio waveform. These audio embeddings are detached from the GPU with the **detach()** function and stored as NumPy arrays in *.npy* files.

P. Video Embedding with LLM2Vec

The first step consists of loading the VisionEncoderDecoderModel for image captioning with LLM2Vec, an upgraded language model to generate embeddings. The **generate caption()** function resizes each of the frames of a video to 224x224, extracts features, and generates a textual caption. Assuming that in the video dataset, each folder constitutes one video, the **load video frames()** function retrieves *.png* images from each folder. Thus, for obtaining an embedding, a caption for every frame is created and sent to LLM2Vec.

These embeddings, after conversion to a NumPy array, are saved in video_embeddings.npy.

Q. Multimodal Fusion

Once we have embeddings for text (T), audio (A), and video (V), we combine them in a gated fusion mechanism. This mechanism gives each modality learnable weights, enabling the model to put more weight on informative features:

$$F = \alpha \cdot T + \beta \cdot A + \gamma \cdot V$$

where α , β , and γ are gate values calculated with a sigmoid function of a learned weight matrix. The gate normalization guarantees that the contributions add up to one:

$$\alpha = \frac{\sigma(w_t)}{\sigma(w_t) + \sigma(w_a) + \sigma(w_v)}$$

with analogous equations for β and γ .

This fusion method enables the model to adaptively control the contribution of every modality, enhancing the overall representation for toxicity detection.

R. Classification

S. Model Architecture and Training

The combined multimodal embedding F is then passed through a classifier. We use a Random Forest classifier in our experiment, as it was found to be highly effective in our earlier experiments. We use SMOTE again before training to balance the data by creating synthetic instances for the minority classes.

T. Evaluation Metrics

We evaluate our classifier using metrics such as accuracy, precision, recall, F1-score, and confusion matrices. Cross-validation (using 5-fold stratified KFold) is employed to ensure the robustness of our model's performance.

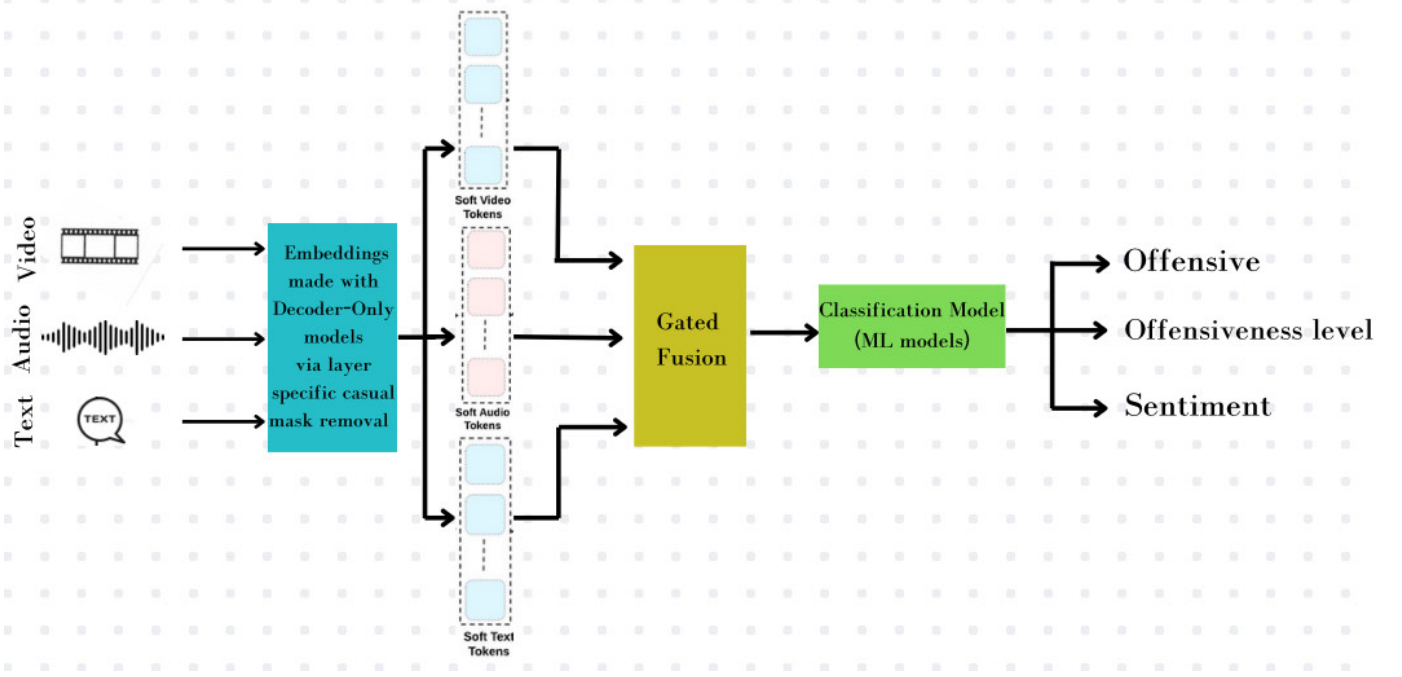


Fig. 2. Methodology to evaluate the LLM2Vec framework

V. EXPERIMENTS AND RESULTS

A. Experimental Setup

The experiments were performed on Kaggle environment with PyTorch, LLM2Vec, MLM2Vec, and Librosa on two NVIDIA T4 GPUs with 16GB RAM.

To achieve reproducibility and efficiency, we preprocessed the data by means of text input normalization, MFCC feature extraction for audio, and video caption generation using a state-of-the-art captioning model. Processing for each modality was carried out independently before generating the embeddings and fusion.

B. Cross Validation and Model Hyperparameters

The train-test split considered for the classification task was **8: 2**. To train the machine learning models, Stratified K-Fold Cross validation was used, with the dataset being split into 5 parts. Each split contains the uniform distribution of the target classes. Before each split, the dataset is shuffled so that the model does not overfit.

During the evaluation of both the frameworks, a Random Forest classifier with 100 estimators was taken to learn the embeddings' features and perform the classification. The model required a minimum of **two** samples to split each node, and each node had at least **one** sample. While performing the classification, another point step that was taken was that - the model classified the data while keeping the class weights balanced, alongside the applied *SMOTE* method, to overcome the class imbalance in the dataset. Additionally, Random Forest classifiers being an *ensemble* method, can avoid overfitting compared to other classifiers. They train faster on moderate-

sized datasets than transformers, sparing us time and compute power, making them an efficient alternative.

C. Embedding Generation

D. LLM2Vec

- **Text:** LLM2Vec was used to generate text embeddings, effectively capturing semantic relationships.
- **Audio:** MFCC features were extracted from audio files, converted to text representations, and embedded using LLM2Vec.
- **Video:** Captions were generated from the video frames using an automated captioning model and then vectors were processed using LLM2Vec to obtain the embeddings.

E. MLM2Vec

- **Text:** A decoder-only model was used to generate text embeddings and stored alongside their respective video split labels.
- **Audio:** The audio waveform was sampled at 16000Hz and then processed using the Wav2Vec2 Model, following which the audio embeddings were combined with their respective labels.
- **Video:** Each video was converted into frames. The embedding obtained from each frame by passing them through the MobileViT model was stored into a matrix, which was then summed along the row axes to obtain the RGB channel information. These matrices were flattened to obtain the individual video split embedding.

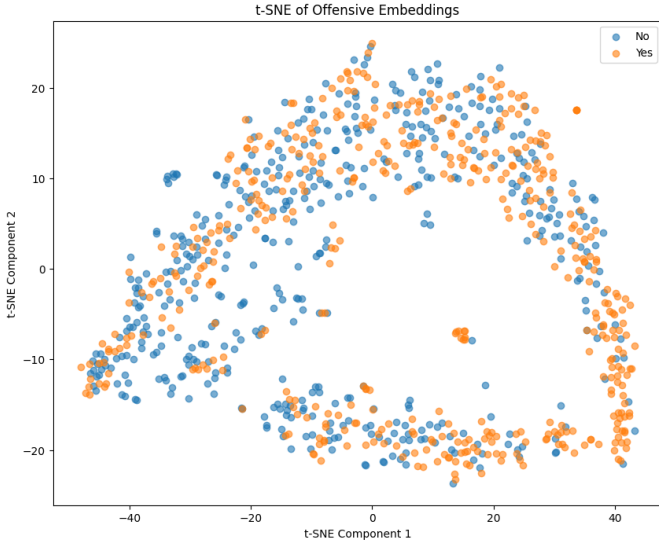


Fig. 3. t-SNE MLM embeddings for *Offensive* category

- The embeddings that were learned were combined using the gated fusion mechanism to obtain a multimodal understanding.
- Our proposal intends to investigate which framework gives us better results, using bidirectional attention or without, allowing us to make further developments in sentiment analysis and toxicity detection.

F. Gated Fusion and Classification

To effectively use multimodal information, a trainable gating component was used. The component controls the contribution of each modality based on the task. The learned embeddings were then used for training and testing machine learning models.

We tried many classification methods and ended up with a Random Forest classifier because it has been widely reported to work well on classification tasks. We trained the classifier using embeddings of LLM2Vec and MLM2Vec with the hyperparameters optimized for achieving maximum accuracy. We tested every model on whether it could correctly detect toxicity and measure sentiment in different data distributions.

The incorporation of a bidirectional attention mechanism in MLM2Vec greatly improved information retention and context accuracy, with superior classification results compared to LLM2Vec. Removing the causal mask in MLM2Vec also improved the ability to understand sophisticated linguistic patterns, with spectacular improvements in toxicity detection.

The t-SNE plots of the final embeddings obtained for each of the categories (offensive, offensiveness level, and sentiment) were plotted and they helped us understand how the data was distributed class-wise.

G. Performance Metrics

The performance metrics - accuracy, precision, recall, and f1-score are used to measure the performance of the frame-

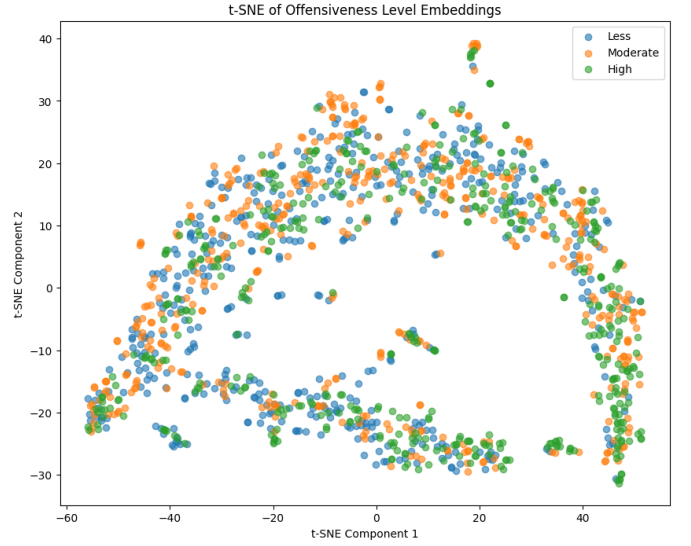


Fig. 4. t-SNE MLM embeddings for *Offensiveness Level* category

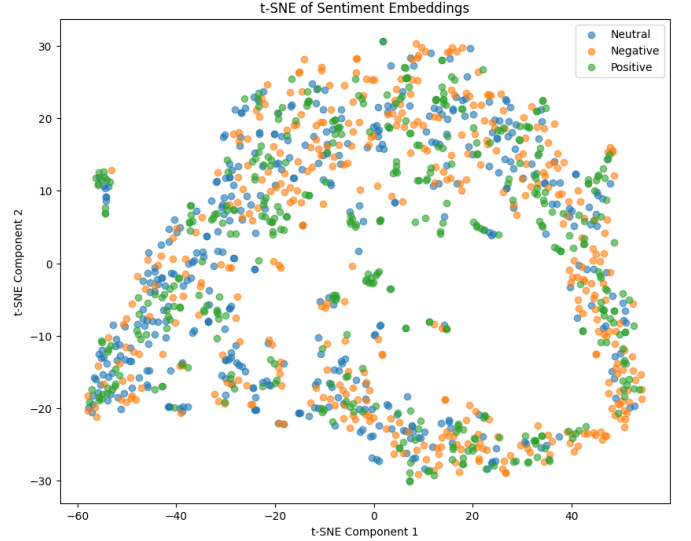


Fig. 5. t-SNE MLM embeddings for *Sentiment* category

works. Each model's performance is shown separately in their respective tables.

H. LLM2Vec Performance

TABLE I entails the results obtained with the LLM2Vec framework. As seen, it gives highest scores in the offensiveness detection.

TABLE I
PERFORMANCE METRICS FOR CLASSIFICATION TASKS USING LLM2Vec

Task	Accuracy (%)	Precision	Recall	F1-Score
Offensiveness Detection	0.85	0.86	0.85	0.84
Offensiveness Level Classification	0.74	0.76	0.74	0.70
Sentiment Classification	0.74	0.72	0.74	0.70

I. MLM2Vec Performance

TABLE II describes the performance metrics obtained for the proposed MLM2Vec framework. This shows that the framework gives its best in offensiveness level classification.

TABLE II
PERFORMANCE METRICS FOR CLASSIFICATION TASKS USING MLM2Vec

Task	Accuracy (%)	Precision	Recall	F1-Score
Offensiveness Detection	0.72	0.72	0.72	0.72
Offensiveness Level Classification	0.79	0.793	0.796	0.793
Sentiment Classification	0.74	0.733	0.74	0.736

TABLE III
COMPARISON OF THE PERFORMANCE METRICS FOR ALL FRAMEWORKS

Framework	Toxicity		Severity		Sentiment	
	F1	Acc	F1	Acc	F1	Acc
LLM2Vec [7]	0.84	0.85	0.70	0.74	0.70	0.74
MLM2Vec	0.72	0.72	0.79	0.79	0.73	0.74
ToxicVidLM (base) [5]	0.88	0.88	0.78	0.78	0.75	0.75

The results obtained with both the frameworks are compared to with that of the base paper [5]. From **TABLE III**, we can see that the LLM2Vec framework is able to give performance close to that of the base work, in the task of *toxicity detection*. However, in the task of finding its severity, the MLM2Vec framework gives the best performance. This shows us that, MLM2Vec outperforms the LLM2Vec and ToxicVidLM frameworks in the nuanced understanding of the sentence (accuracy of **0.79** over **0.72** and **0.74**), but falls short when used for classifying *toxicity* or *sentiment*. The LLM2Vec and ToxicVidLM frameworks give the best performance on the more abstract tasks of sentiment and toxicity classification (accuracies of **0.88** and **0.75** respectively).

Another point to keep in mind, is that MLM2Vec comprises of lightweight models that are more suited to deployment. This is suitable to those, who wish to deploy the solution in a low resource setting. Our proposed MLM2Vec framework allows us to detect the severity of a toxic utterance while limiting its resource usage. It is able to give better or comparable performance with regards to the base framework.

VI. DISCUSSION

Our findings show that LLM2Vec performs better than MLM2Vec on the toxicity classification task, illustrating the utility of bidirectional attention and eliminating the causal mask. Another observation we made was that MLM2Vec performs on par or better than LLM2Vec and ToxicVidLM in the nuanced tasks of severity and sentiment classification. In this section we compare the performance of both these frameworks on the three tasks at hand - offensiveness detection, offensiveness level classification and sentiment analysis. Each of their performance were compared as seen in **TABLE III**.

Considering the task of toxicity classification, LLM2Vec significantly outperforms MLM2Vec. It has much higher accuracy, showing that this framework can be used for higher-

level classification. This happens because the larger pretrained knowledge that LLM2Vec could leverage and the removal of the causal mask, which allowed all the layers to learn contextual information suited to this task.

The performance of LLM2Vec and MLM2Vec in the task of sentiment analysis can be reviewed in more detail, using **TABLES I** and **II**. When it comes to the fine-grained classification (severity and sentiment classification) MLM2Vec performs relatively better than LLM2Vec, which was possible due to its masked language training.

This can be an indicator that the presence of the causal mask allows the model to understand toxic utterances with depth. It is also able to perform comparably with ToxicVidLM in the task of sentiment classification. This shows us the potential of decoder only models, and the effect of the causal masking on their assigned tasks.

This demonstrates the capability of MLM2Vec to scale and perform nuanced multimodal toxic content analysis in resource constrained environments. This is more suited to real-world deployments.

In addition, we noted that the gated fusion mechanism was a key in finding the equilibrium of contributions between text, audio, and video embeddings. The mechanism efficiently recorded multimodal interaction by adjusting each modality dynamically with weights effectively, which lead to an overall performance improvement in classification.

Future research will investigate other embedding methods and fusion approaches to continue improving toxicity detection and sentiment analysis. Further experiments on bigger datasets and practical applications will further establish the robustness and generalizability of our approach.

VII. CONCLUSION AND FUTURE WORK

We have successfully developed a system based on multi-modal fusion and decoder-only models to detect toxicity in code-mixed video content. Our initial results demonstrate the potential of our approach; thus, we are committed to enhance the accuracy and stability of our system further. The results also describes the effect of causal masking used for different tasks in toxicity detection. It showed the LLM2Vec detects the toxicity better (85% accuracy) while MLM2Vec was able to understand the depth of the toxicity and its sentiment (79% and 74% accuracy). MLM2Vec thus gains a deeper understanding of the utterance, with regards to the sentiment and its severity, while keeping its resource usage low and effective. MLM2Vec is also efficient and scalable, making it suited to real world deployments.

Future work will focus on exploring other machine learning classifiers, balancing the class in the dataset, and incorporating more advanced fusion methods. We also plan to examine the decoder-only models' toxicity detection performance and the impact of layer-specific attention mechanisms on this task. This performance of this framework can also be compared with its performance to other lightweight models, especially the small language models, that have gained popularity in their recent years. There is much work to be explored further ahead,

with the ongoing release of various powerful models and their analyses.

Another important direction to consider is the language in which we can perform this task. This framework can be expanded to perform the same task in various Indic languages, if the dataset can be curated with balance so that bias can be avoided. There is a lot of scope for future research.

VIII. REFERENCES

- 1) Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł. and Polosukhin, I., 2017. "Attention is all you need". *Advances in neural information processing systems*, 30.
- 2) Schmidt, A., & Wiegand, M. (2017). A Survey on Hate Speech Detection Using Natural Language Processing. *Proceedings of the Fifth International Workshop on Natural Language Processing for Social Media*.
- 3) Fortuna, P., & Nunes, S. (2018). A Survey on Automatic Detection of Hate Speech in Text. *ACM Computing Surveys*, 51(4), 85.
- 4) Devlin, J., Chang, M.-W., Lee, K., & Toutanova, K. (2019). BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics*.
- 5) Maity, K., Poornash, A.S., Saha, S. and Bhattacharyya, P., 2024. ToxVidLM: A Multimodal Framework for Toxicity Detection in Code-Mixed Videos. *arXiv preprint arXiv:2405.20628*.
- 6) Kiela, D., et al. (2020). The Hateful Memes Challenge: Detecting Hate Speech in Multimodal Memes. *Advances in Neural Information Processing Systems*.
- 7) BehnamGhader, P., Adlakha, V., Mosbach, M., Bahdanau, D., Chapados, N. and Reddy, S., 2024. Llm2vec: Large language models are secretly powerful text encoders. *arXiv preprint arXiv:2404.05961*.
- 8) Baevski, A., Zhou, H., Mohamed, A., & Auli, M. (2020). Wav2Vec 2.0: A Framework for Self-Supervised Learning of Speech Representations. *Advances in Neural Information Processing Systems*.
- 9) Hsu, W.-N., et al. (2021). HuBERT: Self-Supervised Speech Representation Learning by Masked Prediction of Hidden Units. *IEEE Transactions on Audio, Speech, and Language Processing*.
- 10) Dosovitskiy, A., et al. (2020). An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. *International Conference on Learning Representations*.
- 11) Radford, A., et al. (2021). Learning Transferable Visual Models from Natural Language Supervision. *Proceedings of the 38th International Conference on Machine Learning*.
- 12) Baltrušaitis, T., Ahuja, C., & Morency, L.-P. (2018). Multimodal Machine Learning: A Survey and Taxonomy. *IEEE Transactions on Pattern Analysis and Machine Intelligence*.
- 13) Dukić, D. and Šnajder, J., 2024. Looking right is sometimes right: Investigating the capabilities of decoder-only llms for sequence labeling. *arXiv preprint arXiv:2401.14556*.
- 14) R. Singh, R. Kashyap and V. Sharma, Toxic Comment Analyzer using BERT: A Deep Learning Approach for Toxicity Detection, *2023 Second International Conference on Informatics (ICI), Noida, India, 2023, pp. 1-6*
- 15) Mehta, S. and Rastegari, M., 2021. Mobilevit: light-weight, general-purpose, and mobile-friendly vision transformer. *arXiv preprint arXiv:2110.02178*.
- 16) Ericsson, L., Gouk, H., Loy, C.C. and Hospedales, T.M., 2022. Self-supervised representation learning: Introduction, advances, and challenges. *IEEE Signal Processing Magazine*, 39(3), pp.42-62.
- 17) Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S. and Bikel, D., 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*.
- 18) Brown, T., Mann, B., Ryder, N., Subbiah, M., Kaplan, J.D., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A. and Agarwal, S., 2020. Language models are few-shot learners. *Advances in neural information processing systems*, 33, pp.1877-1901.
- 19) Radford, A., Narasimhan, K., Salimans, T. and Sutskever, I., 2018. Improving language understanding by generative pre-training.
- 20) Su, J., Ahmed, M., Lu, Y., Pan, S., Bo, W. and Liu, Y., 2024. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568, p.127063.
- 21) Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P., 2002. SMOTE: synthetic minority over-sampling technique. *Journal of artificial intelligence research*, 16, pp.321-357.
- 22) Walia, Bhumika. (2023). Toxicity in the Digital Age: A Review of Literature on Online Toxic Behaviors and Social Media Platforms. 10.13140/RG.2.2.15541.08165.
- 23) Thara, S. and Poornachandran, P., 2021. Transformer based language identification for malayalam-english code-mixed text. *IEEE Access*, 9, pp.118837-118850.
- 24) Thara, S. and Poornachandran, P., Social media text analytics of Malayalam-English code-mixed using deep learning. *J. Big Data* 9, 45 (2022) [online]
- 25) Sreelakshmi, K., Premjith, B., Chakravarthi, B.R. and Soman, K.P., 2024. Detection of hate speech and offensive language codemix text in dravidian languages using cost-sensitive learning approach. *IEEE Access*, 12, pp.20064-20090.
- 26) Sreelakshmi, K., Premjith, B. and Soman, K.P., 2020. Detection of hate speech text in Hindi-English code-mixed data. *Procedia Computer Science*, 171, pp.737-744.
- 27) Peyrard, M., Borges, B., Gligorić, K. and West, R., 2021. Laughing heads: Can transformers detect what makes a sentence funny?. *arXiv preprint arXiv:2105.09142*.

- 28) Ainslie, J., Lee-Thorp, J., De Jong, M., Zemlyanskiy, Y., Lebrón, F. and Sanghai, S., 2023. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. arXiv preprint arXiv:2305.13245.
- 29) Radford, A., Kim, J.W., Xu, T., Brockman, G., McLeavey, C. and Sutskever, I., 2023, July. Robust speech recognition via large-scale weak supervision. In International conference on machine learning (pp. 28492-28518). PMLR.